🏥 Taller Técnico: Arquitectura de Datos en "HealthPulse"

Materia: Arquitectura de Software

Objetivo: Diseñar e implementar una infraestructura de datos modular, cohesiva y con acoplamiento controlado para un entorno de salud.

Herramientas: dbdiagram.io y SQLiteOnline.com (Motor **PostgreSQL**).

📋 1. Contexto del Negocio

La red de clínicas **"HealthPulse"** necesita modernizar su sistema de gestión. El sistema debe permitir el seguimiento de pacientes, la programación de citas y, lo más importante, el monitoreo histórico de signos vitales (telemetría médica). El reto es construir una base de datos donde la información sea fácil de auditar y los módulos (médicos, pacientes, finanzas) no estén "enredados" entre sí.

📝 2. Requerimientos Funcionales

Para el modelado, deben cumplir con las siguientes entidades y reglas:

1. **Pacientes:** Registro de cédula (única), nombre completo, fecha de nacimiento y tipo de sangre.
2. **Personal Médico:** Registro de nombre, tarjeta profesional (única) y especialidad médica.
3. **Citas Médicas:** Una cita vincula a un paciente con un médico en una fecha y hora específica, incluyendo un motivo de consulta.
4. **Historias Clínicas:** Cada cita genera una entrada en la historia clínica con el diagnóstico y el tratamiento recetado.
5. **Monitoreo de Signos Vitales:** Registro automático de lecturas (frecuencia cardíaca, temperatura y fecha/hora) vinculadas a un paciente durante su estancia en la clínica.

🛠️ 3. Fase 1: Modelado Arquitectónico (DBML)

Utilizando dbdiagram.io, traduzca los requerimientos anteriores a lenguaje **DBML**.

Instrucciones de Diseño:

- Identifique las entidades primarias y asegure la **Alta Cohesión**: la tabla de pacientes solo debe contener datos del paciente, no del médico.
- Establezca las relaciones (Ref) necesarias para que el sistema funcione, cuidando de no crear dependencias circulares.

💻 4. Fase 2: Implementación en PostgreSQL (DDL)

En SQLiteOnline, utilizando el motor **PostgreSQL**, escriba el script DDL de creación.

📜 **Reglas de Implementación Senior:**

- **Constraints Independientes:** Queda estrictamente prohibido definir PRIMARY KEY, FOREIGN KEY, UNIQUE o CHECK en la misma línea de la columna. Todas deben declararse al final de la tabla mediante la sentencia CONSTRAINT con nombres claros (ej. pk_pacientes, fk_citas_medico).
- **Gestión de Acoplamiento (Integridad):** Debe configurar las cláusulas de borrado (ON DELETE).
    - *Reto:* Si un médico es retirado del sistema, ¿qué debe pasar con sus citas programadas? ¿Y con las historias clínicas ya escritas? Use su criterio de arquitecto para decidir entre CASCADE, SET NULL o RESTRICT.
- **Tipos de Datos:** Use SERIAL para IDs, VARCHAR para textos y TIMESTAMP para el monitoreo.

🧠 5. Fase 3: Reflexión Arquitectónica

Al finalizar, cada estudiante debe justificar por escrito:

1. **Modularidad:** Si el módulo de **"Monitoreo de Signos"** se vuelve tan grande que requiere su propio servidor, ¿qué tan fácil es separarlo del resto de la base de datos según su diseño?
2. **Acoplamiento:** ¿Qué tabla es el "punto central" del sistema? ¿Qué pasaría si esa tabla sufre un cambio estructural profundo?
3. **Cohesión:** ¿Por qué es un error de diseño poner el diagnóstico médico directamente en la tabla de la Cita Médica en lugar de una tabla de Historia Clínica?

¡TODO DEBE ESTAR DOCUMENTADO EN GIT HUB!